eedi-mining-misconceptions-in-mathematics

5位 Solution | Ebi

Rank	5
Team	Ebi & ktr
Author	Ebi
Topic ID	551391
Version	Japanese Translation

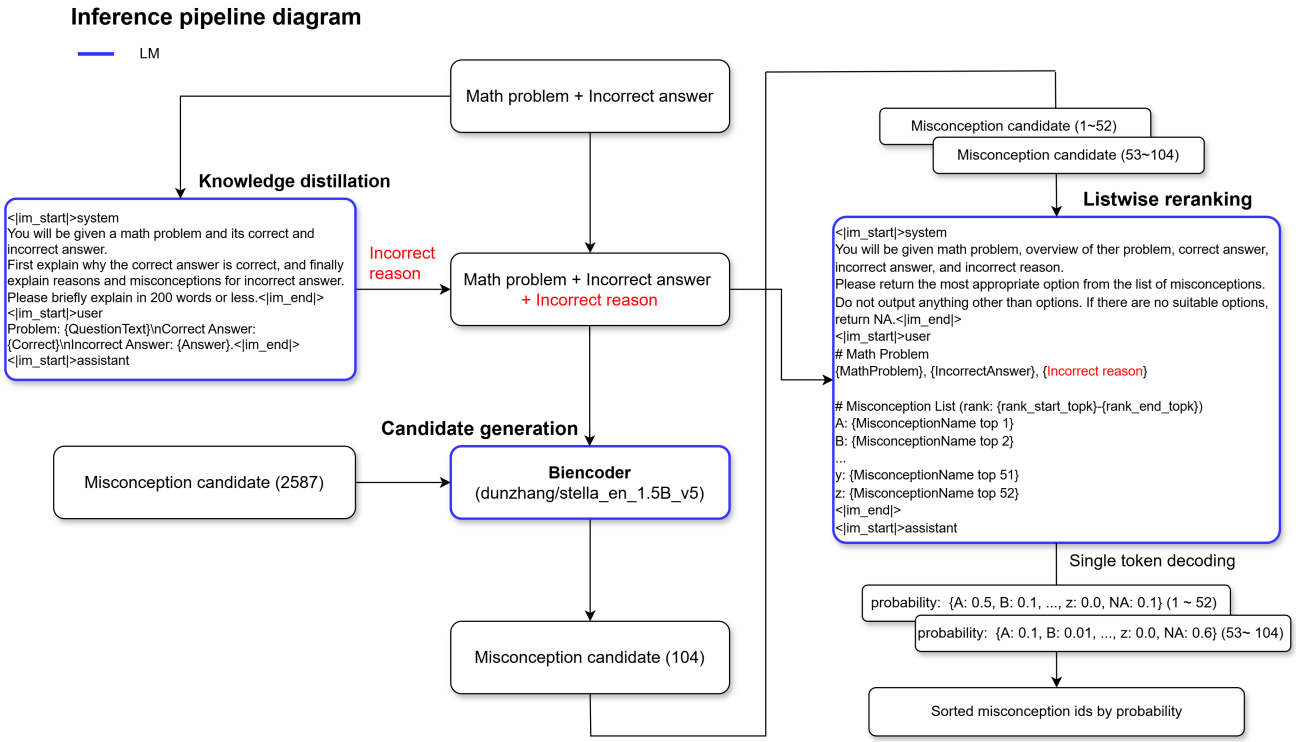
Source: https://www.kaggle.com/competitions/eedi-mining-misconceptions-in-mathematics/discussion/551391
Retrieved with Kaggle CLI on 2026-10-21.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

コメント

まず、工夫の余地が大きく、大きな問題もない、このように興味深いコンペを開催してくださった主催者の皆様に感謝します。また、私にとって初めてのチームアップの経験で、円滑なコミュニケーション環境を作ってくれたチームメイトにも感謝します。

Solutionの概要



1. **合成データ生成**
   - training datasetに含まれない各MisconceptionIdについて、LLMでfew shot promptingを用いて問題を作成します。
     - これらのfew shotのサンプルには、生成対象と類似したMisconceptionNameを持つ問題と誤答を挿入します。
2. **Knowledge Distillation**
   - questiontext、正答、誤答をLLM（Qwen 2.5 32B Instruct）に入力し、誤答の背後にある推論を生成します。この内容は、後続のすべてのmodel（Biencoder、Listwise reranker）の入力として使用します。
3. **候補生成**
   - biencoder modelを用いて、各問題と誤答についてMisconceptionの候補を絞り込みます。

4. **Listwise reranking**
    - Biencoderで並べ替えたMisconception候補を52件ずつ（選択肢として使う大文字と小文字のアルファベットの数に対応）fine-tuningしたLLM（Qwen 2.5 32B Instruct）に入力し、各選択肢の確率を得ます。そして、その確率に基づいてMisconceptionを並べ替えます。
        - この処理には、biencoderの上位104件の候補を使用します。つまり、52件ずつで2回推論を行い、確率を取り出して結果を結合します。
5. **Ensemble**
    - Listwise rerankerはGPTQ + vLLMで推論するため、1回の推論は約120分で済み、3つのfoldの結果をensembleできます。（ただし、こちらで議論されているランダムな遅延のため、私たちのbest subには3つ目のfoldの結果をすべて含めることができませんでした。今後Kaggleのスタッフがこうした問題に対処してくれることを願っています。）

Solutionの詳細

1. **Cross validation**
    - 当初はQuestionIdに基づくGroupKFold(k=5)を使っていましたが、lbがlocal cvより大幅に低かったため、testデータにのみ登場するMisconceptionIdが多数あるのではないかと考えました。これを踏まえ、SubjectIdを用いたGroupKFold(k=5)に切り替えました。この変更により、validationデータにのみ登場するMisconceptionIdの割合が高くなり、cvがlbに近づく傾向がありました。
        - 例 local cv 0.626, lb: 0.633
2. **合成データ生成**
    - 既存の問題がないMisconceptionIdを予測結果に含めるには、合成データ生成が不可欠です。問題がない各MisconceptionNameについて、LLM（gemini-1.5-pro）でfew shot prompting（4 shots）を用いて、既存データセットの情報（QuestionName、SubjectName、ConstructName、Correct Answer、Incorrect Answer）を1セットずつ生成しました。
        - few shot prompting用のサンプルについては、すべてのMisconceptionNameをmodel（dunzhang/stella_en_1.5B_v5）でembeddingし、類似したMisconceptionNameに対応する問題を含めました。これにより、ランダムサンプルを使う場合と比べてcvが約+0.01改善しました。
3. **Knowledge Distillation(KD)**
    - QuestionText、正答、誤答をLLM（Qwen 2.5 32B Instruct）に入力し、Misconceptionを推定します。この情報を使うとbiencoderのcvが+0.04改善します。後述のListwise rerankerはKDと同じmodelを使っていますが、この情報を含めない場合と比べてcvが約+0.01向上しました。
        - prompt

```
PROMPT_FORMAT =
```

4. **候補生成**
    - Biencoderを用いて、各問題と誤答のペアについてMisconceptionName候補を抽出します。negative miningを行っていますが、hard negativeは効果がなかったため、やや緩めのサンプルを含めています。
    - 学習パラメータ
        - library: SentenceTransfomer

        - model: dunzhang/stella_en_1.5B_v5
        - negative mining params
            - range_min: 512
            - num_negatives: 2
        - lora config
            - r: 32
            - lora_alpha: 64
        - loss: CachedMultipleNegativesRankingLoss
        - epoch: 1
    - cv: 0.41, lb: 0.39

5. **Listwise reranker**
    - 概要
        - LLMを用いたrerankingの部分は、このコンペで最も独自性のある部分であり、この部分をどう実装したかがスコアに大きく影響したと考えています。チームメイトの@kcotton21は、コンペの序盤の時点ですでに、単一tokenを生成してそのlogitsを並べ替えに使うというアイデアを思いついていました（参考: [https://arxiv.org/abs/2406.15657)](https://arxiv.org/abs/2406.15657)）。私たちのsolutionはこれに基づいています。
    - prompt
        - さまざまな試行錯誤の末、最終的には以下のpromptを用いて通常のfine-tuningを行い、LLMに適切な選択肢（logits（確率）で並べ替えられるよう、単一tokenである必要があります）を生成させました。

```
NA_PROMPT_FORMAT =
```

        - misconception_list
            - 大文字 + 小文字のアルファベットによる52個の選択肢と、それに対応するMisconceptionNameを含みます。これらはBiencoderの出力順に並んでいます。

```
A: Thinks sign  parallel lines just means opposite
B: Does  recognise the notation  parallel sides
.
.
.
y: Thinks that co-interior angles can lie on opposite sides of a transversal
z: Believes the gradient of perpendicular lines just have opposite signs<|im_end|>
```

        - label_choice
            - fine-tuningに使用する正解の選択肢です。正解が選択肢に含まれていない場合は、"NA"（Qwen 2.5 32Bではこれも単一tokenです）を挿入します。
        - rank
            - Biencoderの出力のうち、どの範囲の選択肢{start_idx - end_idx}が含まれているかを示します。

- 学習
  - biencoderの上位208件の出力を、52件ずつの重複しないグループ（1-52、53-104、105-156、157-208）に分割してfine-tuningを行いました。
  - 学習パラメータ
    - loss
      - 通常のfine-tuningで使われる標準的なcross-entropy lossで、語彙全体にわたって計算します。候補の選択肢 + "NA" tokenのみに限定したcross-entropyを計算しても結果にほとんど差がなかったため、よりシンプルなこの実装方法を選びました。
    - model: Qwen/Qwen2.5-32B-Instruct
    - lora_config
      - r: 24
      - lora_alpha: 48
    - lr: 5e-5
    - scheduler: cosine
    - warmup_steps: 10
    - epoch : 1.0
    - global batch size: 8
    - library: unsloth
- 推論
  - biencoderの出力から上位104件を取り出し、2つのセット（1-52と53-104）に分割します。各セットをmodelに入力し、単一tokenのみを生成して各選択肢の確率を得ます。その後、両セットの結果を結合して並べ替えます。
    - 上位156件（52x3の推論）まで追加するとlocal CVはわずかに改善しましたが、leaderboardのスコアには反映されなかったため、この方法は採用しませんでした。
  - GPTQで4-bitに量子化し、vLLMで推論を行います。

cv(public)[private] scores

	52x1	52x2	52x3
fold 4	0.623(0.627)[0572]	0.626(0.633)[0.581]	0.630(0.631)[0.581]
fold 3		(0.644)[0.581]	
fold (3 + 4)		(0.650)[0.590]	
fold (2(60%sample) + 3 + 4)		(0.653)[0.597]	

計算コスト

私のローカルマシンは1 x RTX 4090です。

32B modelの学習にはA100を（vast.aiから）借りており、実験を含めると合計で約$350かかりました…

32BのListwise modelを1つ学習するのに、A100で約7時間かかりました。

うまくいかなかったこと

- Listwise rerankerで使用する選択肢
  - 単一tokenの選択肢として、アルファベット2文字の組み合わせ、アルファベットと数字0-9の組み合わせ、日本語のひらがなを試しましたが、いずれも52個のアルファベットだけを使う場合より性能が悪化しました。
- QwQ-32B-previewの活用
- 多段階のreranking（Qwen 2.5 7B（104候補） → Qwen 2.5 32B（52候補））
- rerankingのpromptに問題とmisconceptionの例を追加すること

コード公開

学習コードを公開しました。→ Eedi-5th-solution

補足Q&A

Nikita Babych — 補足

おめでとうございます！ public leaderboardで皆さんと競い合えて楽しかったです。

候補生成の段階でmodelをensembleせずに、これほど高いスコアを達成できたのはとても興味深いです。私のsolutionではそれがかなり重要でした。

Ebi — 回答

> 候補生成の段階でmodelをensembleせずに、これほど高いスコアを達成できたのはとても興味深いです。私のsolutionではそれがかなり重要でした。

その理由は、私たちがretrieverの結果を、あなたのように上位50件ではなく上位104件まで32B modelでrerankしたからだと思います。retriever modelの性能は高くなかったものの、高いrecallを得ることができました。

Camaro — 質問

おめでとうございます。solutionときれいなコードを共有してくださりありがとうございます！

データ生成について質問があります。

> これらのfew shotのサンプルには、生成対象と類似したMisconceptionNameを持つ問題と誤答を挿入します。

類似したMisconceptionNameをfew-shotの例として加えるのは巧妙なアプローチですね。ランダムサンプリングと比べてどうだったか確認されましたか。

Ebi — 回答

共有したコードが役に立ったようで嬉しいです。

> 類似したMisconceptionNameをfew-shotの例として加えるのは巧妙なアプローチですね。ランダムサンプリングと比べてどうだったか確認されましたか。

はい、確認しました。local cv、private lb、public lbのすべてで約+0.011~0.014の効果があります。

Camaro — 質問

迅速な回答をありがとうございます！ 試してみます。

さらにいくつか質問があります。

- コードを正しく理解していれば、misconception IDごとに1サンプルを生成しているようです。より多くのサンプルを生成してもスコアは改善しなかったのでしょうか。
- Gemini Flash 1.5を使っていますね。私がこのmodelで生成を試したところ、誤った答えを生成することがありました（misconceptionを捉えられていませんでした）。O1や、少なくともGemini Proなど、他のmodelは試しましたか。

Ebi — 回答

> コードを正しく理解していれば、misconception IDごとに1サンプルを生成しているようです。より多くのサンプルを生成してもスコアは改善しなかったのでしょうか。

試しましたが、local cvが悪化したのでsubmitしませんでした。ただ、いくつかのsolutionを読んだところ、training dataに含まれないMisconceptionIdを推薦結果の上位に持ってくることが重要だったようなので、今ではsubmitしていればpublic lbが上がっていたのではないかと思っています。

> Gemini Flash 1.5を使っていますね。私がこのmodelで生成を試したところ、誤った答えを生成することがありました。O1や、少なくともGemini Proなど、他のmodelは試しましたか。

この点は申し訳ありません。コードの共有時にミスをしており、実際にはgemini-1.5-proを使っていました。gemini-1.5-flashとgemini-1.5-proでbiencoderの精度を比較しましたが、スコアはまったく変わりませんでした（local cvとpublic LBの両方）。

そのため、合成データ生成に使うLLMを変える実験はそこでやめましたが、few-shotの例を変える実験をした際もbiencoderのスコアに大きな変化はなかったことから、今ではListwise rerankingには効果があったかもしれないと考えています。

john — 質問

おめでとうございます。solutionとコードを共有してくださりありがとうございます！ コードがとてもよく構造化されていますね！

retrieverの学習コードについていくつか質問があります。

1. exp_gpu.yamlのhard_negative_params

   range_min: 512, num_negatives: 2

なぜrange_minがこれほど大きいのでしょうか。また、negativeが2件だけなのはなぜですか。なぜそのように決めたのか教えていただけますか。

2. train_args

   num_train_epochs: 1.0, per_device_train_batch_size: 640

   これほど大きなbatch sizeで1 epochだけなのですか。こちらもなぜそのように決めたのでしょうか。

Nikhil Tumbde — 質問

こんにちは@ebinan92さん。コードを公開してくださりありがとうございます。また、金メダル獲得おめでとうございます。

量子化について1点確認させてください。re-ranker（32B）はBF16でfine-tuningされているようなので、LoRAの重みをmodelの重みに直接マージしたのだと思います。それで合っていますか。

また、DataCollatorForCompletionOnlyLMではなくDataCollatorForSeq2Seqを使っている理由はありますか。何か見落としているでしょうか。

Ebi — 回答

> 量子化について1点確認させてください。re-ranker（32B）はBF16でfine-tuningされているようなので、LoRAの重みをmodelの重みに直接マージしたのだと思います。それで合っていますか。

はい、そのとおりです。マージ時の性能劣化を避けるため、QLoRAは使っていません。

> また、DataCollatorForCompletionOnlyLMではなくDataCollatorForSeq2Seqを使っている理由はありますか。何か見落としているでしょうか。

unslothでのfine-tuningについては、このissueを参考にDataCollatorForSeq2Seqを使っています。

ご指摘のとおり、DataCollatorForCompletionOnlyLMを使うことも可能だと思います。

Ebi — 回答

簡潔な回答:

いくつかの組み合わせを試した結果、recall@100（bi-encoderで私が主に目標としていた指標）が最も良かったため、これらのパラメータを選びました。

詳しい説明:

- range_min: 512
  - このデータではhard negativeが効果的でなかったため、大きめの値を採用しました。
- num_negatives: 2
  - この値を増やすと学習データは増えますが、2前後で性能が頭打ちになったため、2に設定しました。
- num_train_epochs: 1.0
  - 1より大きい値では過学習したため、1に設定しました。

- per_device_train_batch_size: 640
    - contrastive learningでは一般にbatch sizeが大きいほど良いですが、これ以上増やしても精度の改善はごくわずかだったため、この値のままにしました。